Video-Based Traffic Light Detection

Using Temporal Consistency and YOLOv8s

A Complete Beginner's Study + Project Guide

Prepared for Absolute Beginners

April 25, 2026

Contents

Part I
Foundational Theory

1 What is a Digital Image?

1.1 The Big Idea

Imagine a mosaic painting made from thousands of tiny coloured tiles. From far away it looks like a real picture, but up close you see individual squares. A digital image works exactly the same way. Each tiny square is called a **pixel** (short for *picture element*).

1.2 Pixels and RGB Colour

Every pixel stores a colour. Computers represent colour using three numbers: **Red**, **Green**, and **Blue** (RGB). Each number ranges from 0 (none of that colour) to 255 (maximum of that colour).

- **(255, 0, 0)** = bright red
- **(0, 255, 0)** = bright green
- **(255, 255, 0)** = yellow (red + green mixed)
- **(0, 0, 0)** = black
- **(255, 255, 255)** = white

1.3 Resolution

Resolution is the number of pixels in width × height. A 640×480 image has $640 \times 480 = 307{,}200$ pixels. Higher resolution = more detail = larger file size.

1.4 Video as a Sequence of Frames

A video is just many images (called **frames**) shown very fast, one after another. At 30 **FPS** (frames per second), 30 individual images are displayed every second, creating the illusion of motion, like a flipbook cartoon.

2 What is Computer Vision?

2.1 Definition

Computer Vision (CV) is the field of giving computers the ability to "see" and understand images or videos, just like humans do with their eyes and brain.

2.2 Real-World Applications

- Self-driving cars detecting pedestrians and traffic lights
- Face unlock on smartphones
- Medical imaging (detecting tumours in X-rays)
- Quality control in factories (spotting defective products)
- Security cameras that alert when a person enters a restricted area

2.3 What Problem Does It Solve?

Humans naturally understand an image instantly. Computers only see numbers. CV bridges that gap by teaching algorithms to extract *meaning* from pixel arrays.

3 What is Object Detection?

3.1 Classification vs. Detection

- **Image Classification**: "Is there a cat in this image?" → Yes/No answer.
- **Object Detection**: "Where is the cat, and where are all the dogs?" → Location + label for *each* object.

3.2 Bounding Boxes

Detection draws a **bounding box** (a rectangle) around each detected object. The box is described by four numbers: (x, y, w, h) where (x, y) is the centre of the box, w is its width, and h is its height (all normalised to 0–1 relative to image size in YOLO format).

3.3 Analogy

Think of a "Where's Wally?" puzzle. Classification says "Wally is in the picture". Detection says "Wally is in the top-right corner, inside this rectangle".

4 What is a Neural Network?

4.1 Inspiration from the Brain

The human brain contains roughly 86 billion **neurons** (nerve cells). Each neuron receives signals, processes them, and sends an output to other neurons. A **Neural Network** (NN) is a mathematical model inspired by this idea.

4.2 Structure

A basic neural network has three types of layers:

1. **Input Layer**: receives raw data (e.g. pixel values).
2. **Hidden Layers**: perform transformations, extract features.
3. **Output Layer**: produces the final answer (e.g. class probabilities).

4.3 Weights and Learning

Each connection between neurons has a **weight** — a number saying how important that connection is. During training, the network adjusts weights to minimise errors. This process, called **backpropagation**, is like a student correcting their answers after getting exam results.

4.4 Analogy

Imagine learning to recognise apples. At first you guess randomly. After seeing thousands of labelled examples ("this is an apple", "this is not"), you gradually learn the patterns. Neural networks do the exact same thing, just with numbers instead of experience.

4.5 Simple Math Intuition

Each neuron computes:

$$\text{output} = \text{activation}\left(\sum_i w_i \cdot x_i + b\right)$$

where x_i are inputs, w_i are weights, b is a bias (a fixed offset), and *activation* is a function that introduces non-linearity (e.g. ReLU: $f(x) = \max(0, x)$).

5 What is a Convolutional Neural Network (CNN)?

5.1 The Problem with Normal Networks on Images

A 640×640 RGB image has $640 \times 640 \times 3 = 1{,}228{,}800$ values. Feeding all of these into a regular neural network is computationally exploding. CNNs solve this.

5.2 Filters (Kernels)

A **filter** is a small matrix (e.g. 3×3) of weights. It slides across the image computing a dot product at each position. This produces a **feature map** — a new image highlighting a specific pattern (e.g. edges, corners, colours).

5.3 Analogy: The Magnifying Glass

Imagine sliding a tiny magnifying glass across a newspaper to look for certain patterns — first scan for horizontal lines, then vertical lines, then curves. Each scan produces a map of where that pattern occurs. Multiple filters detect multiple patterns simultaneously.

5.4 Pooling

After convolution, **pooling** reduces the feature map size by summarising small regions (e.g. taking the maximum value in each 2×2 block). This reduces computation and makes detection position-tolerant.

5.5 CNN Layers Summary

Layer	Operation	Output
Convolution	Slide filter, compute dot products	Feature map
Activation (ReLU)	Set negatives to zero	Non-linear feature map
Pooling	Downsample by max/average	Smaller feature map
Fully Connected	All neurons connected	Class scores / box coords

6 What is YOLO?

6.1 The Core Idea

YOLO (You Only Look Once) is an object detection algorithm. The name describes its key innovation: it processes the entire image *once* through the network and predicts all bounding boxes and classes simultaneously.

6.2 Older Approach vs. YOLO

Older methods (like R-CNN) first proposed 2000 candidate regions, then classified each one separately — very slow. YOLO treats detection as a single regression problem, making it fast enough for real-time video.

6.3 How YOLO Works — Intuition

1. Divide the image into an $S \times S$ grid (e.g. 20×20).
2. Each grid cell predicts: bounding box coordinates, confidence score, and class probabilities.
3. After prediction, apply **Non-Maximum Suppression** (NMS) to remove duplicate boxes, keeping only the highest-confidence box per object.

6.4 Speed vs. Accuracy Trade-off

YOLO prioritises speed. It may miss some small objects that slower, more thorough methods catch, but it runs at 30–100+ FPS on modern hardware, making it ideal for real-time video.

7 What is YOLOv8s Specifically?

7.1 YOLOv8 Model Family

YOLOv8 (released by Ultralytics in 2023) is the 8th major version of YOLO. It comes in five sizes:

Model	Parameters	mAP	Speed
YOLOv8n (nano)	3.2M	Lowest	Fastest
YOLOv8s (small)	**11.2M**	**Medium**	**Fast**
YOLOv8m (medium)	25.9M	Good	Medium
YOLOv8l (large)	43.7M	Better	Slow
YOLOv8x (extra-large)	68.2M	Best	Slowest

We use **YOLOv8s** because it balances accuracy and speed well for a traffic light detection project on standard hardware.

7.2 What YOLOv8s Outputs

For each detected object, the model outputs:

- **Bounding box**: $(x_{\text{centre}}, y_{\text{centre}}, w, h)$ normalised to $[0, 1]$.
- **Class label**: e.g. "red", "green", "yellow".
- **Confidence score**: a number between 0 and 1 indicating how certain the model is.

7.3 Architecture Overview

YOLOv8s consists of three main sections:

1. **Backbone (CSPDarknet)**: Extracts features from the input image using stacked convolutional blocks.
2. **Neck (FPN + PAN)**: Merges features from different scales so both large and small objects are detectable.
3. **Head**: Predicts bounding boxes and class probabilities for each grid cell.

8 What is Transfer Learning?

8.1 Definition

Transfer Learning means using a model already trained on a large dataset (e.g. COCO with 80 object classes) as a starting point, then fine-tuning it on your smaller, specific dataset (e.g. traffic lights only).

8.2 Why It Works

The early layers of any CNN learn general patterns: edges, corners, textures. These are useful for *any* image recognition task. Only the later layers need to be re-trained for your specific problem.

8.3 Analogy

Learning to drive is easier if you already know how to ride a bicycle. You already understand balance, steering, traffic rules. You only need to learn the new controls. Transfer learning works the same way — the "general image skills" are already learned.

8.4 Practical Benefit

Without transfer learning, you might need millions of labelled images. With transfer learning, a few hundred or few thousand images can give good results.

9 What is Temporal Consistency in Video?

9.1 The Flickering Problem

When running object detection frame-by-frame on a video, the model may:

- Detect a "red" light in frame 10, then miss it in frame 11, then detect it again in frame 12.
- Predict "red" in frame 5, "green" in frame 6 (impossible — lights don't switch that fast).

This is called **flickering** or **detection noise**.

9.2 Temporal Consistency Solution

Temporal Consistency means using information from *multiple consecutive frames* together to make a more stable, reliable decision.

9.3 Analogy

Imagine watching a blurry CCTV feed. In one frame you think you see a person; in the next you're not sure. If you look at 5 frames together and the person appears in 4 of them, you are much more confident they are real. That's temporal consistency — using time to reduce uncertainty.

10 What is a Sliding Window (Temporal Window)?

10.1 Concept

A **temporal sliding window** keeps track of the last N frames of detections. When a new frame arrives, the oldest frame is removed and the newest is added. The window "slides" forward in time.

10.2 Analogy

Think of your short-term memory. Right now you remember roughly the last 10 seconds of a conversation. As new words are spoken, the oldest ones fade out. You use this recent window to understand context.

10.3 In Code

We store a Python `deque` (double-ended queue) of size N. Each element contains the detections from one frame.

11 What is Confidence Averaging?

11.1 Formula

Given confidence scores $C_t, C_{t-1}, \ldots, C_{t-N+1}$ from the last N frames:

$$\bar{C}_t = \frac{\sum_{i=0}^{N-1} w_{t-i} \cdot C_{t-i}}{\sum_{i=0}^{N-1} w_{t-i}}$$

where w_{t-i} is the weight assigned to the detection i frames ago (recent frames get higher weight).

11.2 Simple Unweighted Case

If all weights are equal ($w = 1$):

$$\bar{C}_t = \frac{1}{N} \sum_{i=0}^{N-1} C_{t-i}$$

11.3 Analogy

A weather forecaster doesn't just use today's reading to predict rain. They average the last 5 days of data, giving more weight to yesterday than last week. Confidence averaging does the same for detection scores.

12 What is Majority Voting?

12.1 Concept

Look at the class label predicted across the last N frames. Choose the label that appears **most often** (the majority). If $N = 5$ and the predictions are [red, red, green, red, red], majority vote = "red".

12.2 Analogy

You ask 5 friends what colour a shirt is. Four say "blue" and one says "purple". You go with blue — majority wins.

12.3 Why It Helps

A single wrong prediction from a noisy frame is outvoted by the consistent predictions from cleaner frames.

13 What is Bounding Box Smoothing?

13.1 The Problem

Even when detection is correct, the bounding box coordinates often jump around slightly between frames (jittering), making the overlay annotation look shaky.

13.2 Moving Average Solution

For each coordinate (e.g. x), compute a moving average over the last N frames:

$$\bar{x}_t = \frac{1}{N} \sum_{i=0}^{N-1} x_{t-i}$$

13.3 Analogy

Imagine drawing a line while sitting in a moving bus. Your hand shakes, so the line is wobbly. Smoothing averages out the shakes, producing a cleaner line.

14 Evaluation Metrics: mAP, Precision, Recall, IoU, FPS

14.1 IoU (Intersection over Union)

IoU measures how well a predicted bounding box matches the ground truth (correct) box:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

IoU = 1 means perfect overlap. IoU = 0 means no overlap. Typically, IoU ≥ 0.5 counts as a correct detection.

14.2 Precision and Recall

First, define:

- **TP** (True Positive): correctly detected object.
- **FP** (False Positive): detected something that isn't there.
- **FN** (False Negative): missed a real object.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Analogy: Precision is "of everything I flagged, how much was actually correct?" Recall is "of everything that was real, how much did I find?"

14.3 mAP (mean Average Precision)

mAP is the primary metric for object detection. It is the mean (average) of **AP** values across all classes.

AP is the area under the Precision–Recall curve. Plotting precision vs. recall at different confidence thresholds gives a curve; the area under it is AP. Higher mAP = better detector.

mAP@0.5 means IoU threshold of 0.5 is used to decide TP/FP.

14.4 FPS (Frames Per Second)

FPS is how many video frames the model can process per second. For real-time video, you need at least 25–30 FPS. Faster models = higher FPS.

15 What is HSV Colour Space?

15.1 RGB Limitations for Colour Detection

RGB mixes colour (hue) and brightness together. If the lighting changes (sunny vs. cloudy), the same red traffic light produces very different RGB values, making colour-based filtering unreliable.

15.2 HSV

HSV (Hue, Saturation, Value) separates:

- **Hue**: the actual colour (0–180° in OpenCV, corresponding to the colour wheel).
- **Saturation**: how vivid/pure the colour is (0 = grey, 255 = vivid).
- **Value**: brightness (0 = black, 255 = bright).

To detect red traffic lights regardless of lighting, you only threshold the Hue channel, which remains relatively stable.

16 What is OpenCV?

16.1 Overview

OpenCV (Open Source Computer Vision Library) is a free Python/C++ library with hundreds of functions for image and video processing.

16.2 What We Use It For in This Project

- Reading video files frame by frame (`cv2.VideoCapture`)
- Resizing images (`cv2.resize`)
- Converting colour spaces (`cv2.cvtColor`)

- Drawing bounding boxes and text on frames (`cv2.rectangle`, `cv2.putText`)
- Writing output video (`cv2.VideoWriter`)
- Displaying frames in a window (`cv2.imshow`)

Part II
Project Architecture

17 Full System Pipeline

The project processes a video through six stages. Below each stage is described with what enters, what happens, and what exits.

17.1 Stage 1: Input Video Stream → Frame Extraction

- **Input**: A video file (e.g. `.mp4`).
- **Process**: OpenCV reads the video and extracts frames one at a time using `VideoCapture.read()`.
- **Output**: Individual BGR image arrays, shape $(H, W, 3)$.
- **Analogy**: Splitting a flipbook into individual pages.

17.2 Stage 2: Image Preprocessing

- **Input**: A raw video frame (arbitrary size).
- **Process**: Resize to 640×640 (YOLOv8 input size). Normalise pixel values from $[0, 255]$ to $[0.0, 1.0]$. This is handled internally by the Ultralytics library.
- **Output**: Normalised $640 \times 640 \times 3$ tensor.
- **Analogy**: Before scanning a document, you fit it onto an A4 page and adjust the contrast.

17.3 Stage 3: YOLOv8s Object Detection

- **Input**: Preprocessed image tensor.
- **Process**: YOLOv8s backbone extracts features; the head predicts box coordinates and class scores; NMS removes duplicates.
- **Output**: List of detections — each containing: class label, confidence score, bounding box (x, y, w, h).
- **Analogy**: A security guard scanning a crowd, tagging each person of interest with a name badge and a certainty rating.

17.4 Stage 4: Temporal Verification Module (TVM)

The TVM has four sub-components:

17.4.1 4a. Sliding Window Storage

- **Input**: New frame's detections.
- **Process**: Append detections to a deque of length N (e.g. $N = 5$). Remove oldest if full.
- **Output**: Window containing detections from the last N frames.

17.4.2 4b. Bounding Box Smoothing

- **Input**: N sets of box coordinates.
- **Process**: Compute element-wise mean of (x, y, w, h) across all frames in the window.
- **Output**: One smoothed bounding box.

17.4.3 4c. Confidence Averaging

- **Input**: N confidence scores.
- **Process**: Apply weighted average (recent frames weighted more highly).
- **Output**: One averaged confidence score.

17.4.4 4d. Majority Voting

- **Input**: N class labels.
- **Process**: Count occurrences of each class; return the most frequent.
- **Output**: One stable class label.

17.5 Stage 5: Final Stable Output

- **Input**: Smoothed box, averaged confidence, voted class label.
- **Process**: Draw annotated bounding box and label on the original frame. Write to output video.
- **Output**: Annotated video saved to disk.

Part III
Environment Setup

18 Opening a Terminal

A **terminal** (also called command prompt or shell) is a text-based interface where you type commands to control your computer.

- **Windows**: Press `Win + R`, type `cmd`, press Enter. Or search "Anaconda Prompt" if Anaconda is installed.
- **Mac**: Press `Cmd + Space`, type `Terminal`, press Enter.
- **Linux**: Press `Ctrl + Alt + T`.

19 Installing Python 3.10

1. Go to https://www.python.org/downloads/
2. Download Python 3.10 for your operating system.
3. **Windows**: During installation, tick "Add Python to PATH" before clicking Install.
4. Verify installation in terminal:

```
python --version
# Expected output: Python 3.10.x
```

20 Creating a Virtual Environment

A **virtual environment** is an isolated Python workspace. Libraries installed here do not interfere with other projects.

```
# Navigate to where you want your project
cd Documents

# Create a folder for the project
mkdir traffic_light_project
cd traffic_light_project

# Create a virtual environment named 'venv'
python -m venv venv

# Activate it (Windows)
venv\Scripts\activate

# Activate it (Mac / Linux)
source venv/bin/activate
```

```

# Your terminal prompt should now start with (venv)
```

21 Installing Required Libraries

```
# Upgrade pip (Python's package installer) first
pip install --upgrade pip

# Install PyTorch (the deep learning framework)
pip install torch torchvision

# Install Ultralytics (provides YOLOv8)
pip install ultralytics

# Install OpenCV (image/video processing)
pip install opencv-python

# Install NumPy (numerical arrays)
pip install numpy

# Install Matplotlib (graphs and plots)
pip install matplotlib
```

21.1 Verifying Installation

```
# Save this as verify.py and run: python verify.py
import torch
import cv2
import numpy as np
import matplotlib
from ultralytics import YOLO

print("PyTorch version:", torch.__version__)
print("OpenCV version:", cv2.__version__)
print("NumPy version:", np.__version__)
print("GPU available:", torch.cuda.is_available())

model = YOLO("yolov8s.pt")  # downloads weights automatically
print("YOLOv8s loaded successfully!")
```

22 Project Folder Structure

```
traffic_light_project/
|
|-- data/
|   |-- images/
```

```
|   |   |-- train/      # Training images
|   |   |-- val/        # Validation images
|   |   |-- test/       # Test images
|   |-- labels/
|   |   |-- train/      # YOLO label files for training images
|   |   |-- val/        # YOLO label files for validation images
|   |   |-- test/       # YOLO label files for test images
|   |-- data.yaml       # Dataset configuration file
|
|-- models/
|   |-- yolov8s.pt      # Pre-trained weights (downloaded
    automatically)
|   |-- best.pt         # Your trained weights (created after
    training)
|
|-- scripts/
|   |-- train.py        # Training script
|   |-- detect.py       # Inference / demo script
|   |-- tvm.py          # Temporal Verification Module
|   |-- evaluate.py     # Evaluation script
|
|-- output/
|   |-- output_video.mp4    # Result video
|   |-- plots/              # Precision-recall curves, etc.
|
|-- venv/               # Virtual environment (do not edit
    manually)
```

Part IV
Dataset Preparation

23 Finding Traffic Light Datasets

The following publicly available datasets are suitable for this project:

1. **LISA Traffic Light Dataset** — https://www.kaggle.com/datasets/mbornoe/lisa-traffic-light-dataset
   Contains US traffic lights with annotations. Large and high quality.

2. **Roboflow Traffic Light Dataset** — https://universe.roboflow.com/search?q=traffic+light
   Search for "traffic light" and filter by YOLO format. Many ready-to-use datasets available.

3. **Bosch Small Traffic Lights Dataset** — https://hci.iwr.uni-heidelberg.de/content/bosch-small-traffic-lights-dataset
   High-quality dataset with small traffic lights, useful for fine-grained detection.

24 YOLO Label Format

Each image has a corresponding `.txt` label file with the same name (e.g. `image001.jpg` → `image001.txt`).

Each line in the label file represents one object:

`class_id` `x_center` `y_center` `width` `height`

All values except `class_id` are **normalised** to $[0, 1]$ (divided by image width/height).

24.1 Example

For an image 640×480 with a red traffic light bounding box at pixel coordinates $(200, 100, 50, 80)$ (x, y, w, h):

$$x_c = \frac{200}{640} = 0.3125, \quad y_c = \frac{100}{480} = 0.2083, \quad w = \frac{50}{640} = 0.0781, \quad h = \frac{80}{480} = 0.1667$$

Label line (assuming class 0 = red):

```
0 0.3125 0.2083 0.0781 0.1667
```

25 Dataset Split

Split images into:

- **Train (70%)**: used to train the model.
- **Validation (20%)**: used during training to check progress (not used for weight updates).
- **Test (10%)**: used only at the end to report final results.

```
import os, shutil, random

# Point to your full image folder
src_images = "all_images/"
src_labels = "all_labels/"

splits = {"train": 0.7, "val": 0.2, "test": 0.1}
images = os.listdir(src_images)
random.shuffle(images)

n = len(images)
train_end = int(n * 0.7)
val_end = train_end + int(n * 0.2)

for split, files in zip(
    ["train", "val", "test"],
```

```
    [images[:train_end], images[train_end:val_end], images[
        val_end:]]
):
    os.makedirs(f"data/images/{split}", exist_ok=True)
    os.makedirs(f"data/labels/{split}", exist_ok=True)
    for f in files:
        shutil.copy(f"{src_images}/{f}", f"data/images/{split}/{f
            }")
        label = f.replace(".jpg", ".txt")
        shutil.copy(f"{src_labels}/{label}", f"data/labels/{split
            }/{label}")

print("Dataset split complete.")
```

26 Writing the data.yaml File

```
# data/data.yaml

path: data                      # Root path of the dataset
train: images/train             # Relative path to training images
val: images/val                 # Relative path to validation images
test: images/test               # Relative path to test images

nc: 3                           # Number of classes

names:                          # Class names (must match label file
    class IDs)
  0: red
  1: green
  2: yellow
```

Part V
Full Code Implementation

27 train.py — Training Script

```
# train.py
# Purpose: Fine-tune YOLOv8s on our traffic light dataset.

from ultralytics import YOLO  # Import the YOLOv8 library

def main():
    # Load the pre-trained YOLOv8s model.
    # 'yolov8s.pt' will be downloaded automatically on first run.
    # This gives us transfer learning     weights already know
        general features.
```

```
    model = YOLO("yolov8s.pt")

    # Start training.
    results = model.train(
        data="data/data.yaml",    # Path to dataset config file
        epochs=50,                # Number of full passes through training data
        imgsz=640,                # Resize all images to 640x640 during training
        batch=16,                 # Process 16 images at a time (reduce if out of memory)
        lr0=0.01,                 # Initial learning rate (how big each weight update is)
        patience=10,              # Stop early if no improvement for 10 epochs
        device="0",               # Use GPU 0 (use "cpu" if no GPU available)
        name="traffic_light_v1", # Folder name for saving results
        project="models",         # Parent folder for all training runs
        pretrained=True,          # Use the pre-trained weights (transfer learning)
        verbose=True,             # Print progress to terminal
    )

    # The best weights are saved automatically at:
    # models/traffic_light_v1/weights/best.pt
    print("Training complete!")
    print(f"Best mAP@0.5: {results.results_dict['metrics/mAP50(B)']:.4f}")

if __name__ == "__main__":
    main()
```

28 tvm.py — Temporal Verification Module

```
# tvm.py
# Purpose: Stabilise detections across consecutive video frames.

from collections import deque    # A list that auto-removes oldest items when full
import numpy as np               # For numerical operations


class TemporalVerificationModule:
    """
    Stores detections from the last N frames and provides:
    - Smoothed bounding boxes (reduces jitter)
    - Averaged confidence scores (reduces noise)
    - Majority-voted class label (reduces flickering)
```

```
    """

    def __init__(self, window_size=5):
        # window_size: how many past frames to remember
        self.window_size = window_size

        # deque with maxlen automatically removes oldest entry when full
        self.window = deque(maxlen=window_size)

        # Decay weights: more recent frames get higher weight.
        # For window_size=5: weights = [1,2,3,4,5]  normalised to sum=1.
        raw_weights = np.arange(1, window_size + 1, dtype=float)
        self.weights = raw_weights / raw_weights.sum()

    def add_detection(self, detection):
        """
        Add the best detection from the current frame to the window.

        detection: dict with keys:
            'class'     -> string, e.g. 'red'
            'confidence' -> float, e.g. 0.87
            'box'       -> list [x, y, w, h] normalised to [0,1]
        """
        self.window.append(detection)

    def get_smoothed_box(self):
        """
        Compute the mean bounding box across all frames in the window.
        Returns [x, y, w, h] or None if window is empty.
        """
        if not self.window:
            return None  # Nothing to smooth yet

        # Stack all boxes into a 2D array of shape (N, 4)
        boxes = np.array([d['box'] for d in self.window])

        # Compute element-wise mean across rows (across frames)
        return boxes.mean(axis=0).tolist()  # Returns [x, y, w, h]

    def get_averaged_confidence(self):
        """
        Compute weighted average of confidence scores.
        Recent frames contribute more (higher weights).
        """
        if not self.window:
            return 0.0
```

```
60
61          n = len(self.window)  # Actual number of frames in window
                (may be < window_size)
62
63          # Use the last n weights (in case window isn't full yet)
64          w = self.weights[-n:]
65          w = w / w.sum()  # Re-normalise so weights sum to 1
66
67          # Extract confidence scores from each frame's detection
68          scores = np.array([d['confidence'] for d in self.window])
69
70          # Weighted dot product: sum(w_i * score_i)
71          return float(np.dot(w, scores))
72
73      def get_majority_vote(self):
74          """
75  ␣␣␣␣␣␣␣␣Return␣the␣most␣frequently␣predicted␣class␣label␣in␣the␣
        window.
76  ␣␣␣␣␣␣␣␣Ties␣are␣broken␣by␣the␣first␣maximum␣found.
77  ␣␣␣␣␣␣␣␣"""
78          if not self.window:
79              return None
80
81          # Collect all class labels from the window
82          labels = [d['class'] for d in self.window]
83
84          # Count occurrences of each unique label
85          unique_labels, counts = np.unique(labels, return_counts=
                True)
86
87          # Return the label with the highest count
88          return unique_labels[np.argmax(counts)]
89
90      def get_stable_detection(self):
91          """
92  ␣␣␣␣␣␣␣␣Convenience␣method:␣returns␣all␣three␣stabilised␣outputs␣
        at␣once.
93  ␣␣␣␣␣␣␣␣Returns␣a␣dict␣or␣None␣if␣the␣window␣is␣empty.
94  ␣␣␣␣␣␣␣␣"""
95          if not self.window:
96              return None
97
98          return {
99              'class': self.get_majority_vote(),
100             'confidence': self.get_averaged_confidence(),
101             'box': self.get_smoothed_box()
102         }
```

29 detect.py — Inference and Demo Script

```python
# detect.py
# Purpose: Run YOLOv8s on a video and apply TVM for stable output.

import cv2                          # OpenCV for video I/O and drawing
from ultralytics import YOLO        # YOLOv8 model
from tvm import TemporalVerificationModule  # Our TVM class

#            Configuration


VIDEO_PATH = "input_video.mp4"        # Path to input video
OUTPUT_PATH = "output/output_video.mp4"  # Where to save result
MODEL_PATH = "models/traffic_light_v1/weights/best.pt"  # Trained model weights
CONF_THRESHOLD = 0.4                  # Ignore detections below 40% confidence
WINDOW_SIZE = 5                       # TVM window size (frames to remember)

# Colour map for drawing boxes (BGR format for OpenCV)
COLOURS = {
    "red":    (0, 0, 255),     # Red light     red box
    "green":  (0, 255, 0),     # Green light     green box
    "yellow": (0, 255, 255),   # Yellow light     yellow box
}

def draw_detection(frame, box, label, conf, img_w, img_h):
    """Draw a bounding box with label and confidence on a frame."""

    # Unpack normalised box coordinates
    x_c, y_c, w, h = box

    # Convert from normalised [0,1] to pixel coordinates
    x1 = int((x_c - w / 2) * img_w)
    y1 = int((y_c - h / 2) * img_h)
    x2 = int((x_c + w / 2) * img_w)
    y2 = int((y_c + h / 2) * img_h)

    colour = COLOURS.get(label, (255, 255, 255))  # Default white if unknown class

    # Draw rectangle (bounding box)
    cv2.rectangle(frame, (x1, y1), (x2, y2), colour, thickness=2)

    # Prepare label text
    text = f"{label} {conf:.2f}"
```

```
41
42      # Draw filled rectangle behind text for readability
43      (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX,
            0.6, 2)
44      cv2.rectangle(frame, (x1, y1 - th - 8), (x1 + tw, y1), colour
            , -1)
45
46      # Draw text
47      cv2.putText(frame, text, (x1, y1 - 4),
48                  cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 2)
49
50      return frame
51
52
53  def main():
54      # Load the trained YOLOv8s model
55      model = YOLO(MODEL_PATH)
56
57      # Open the input video
58      cap = cv2.VideoCapture(VIDEO_PATH)
59      if not cap.isOpened():
60          raise FileNotFoundError(f"Cannot␣open␣video:␣{VIDEO_PATH}
                ")
61
62      # Get video properties
63      fps    = cap.get(cv2.CAP_PROP_FPS)
64      width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
65      height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
66      total  = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
67
68      print(f"Video:␣{width}x{height}␣@␣{fps:.1f}␣FPS,␣{total}␣
            frames")
69
70      # Set up output video writer
71      fourcc = cv2.VideoWriter_fourcc(*"mp4v")  # MP4 codec
72      writer = cv2.VideoWriter(OUTPUT_PATH, fourcc, fps, (width,
            height))
73
74      # Create one TVM instance per tracked class
75      tvm_red    = TemporalVerificationModule(window_size=
            WINDOW_SIZE)
76      tvm_green  = TemporalVerificationModule(window_size=
            WINDOW_SIZE)
77      tvm_yellow = TemporalVerificationModule(window_size=
            WINDOW_SIZE)
78      tvm_map = {"red": tvm_red, "green": tvm_green, "yellow":
            tvm_yellow}
79
80      frame_idx = 0  # Track which frame we're on
81
82      while True:
```

```
        ret, frame = cap.read()  # Read next frame; ret=False when video ends
        if not ret:
            break  # No more frames

        frame_idx += 1

        # Run YOLOv8 inference on this frame
        results = model.predict(
            source=frame,
            conf=CONF_THRESHOLD,  # Filter detections below this confidence
            verbose=False          # Don't print per-frame info to terminal
        )

        # results[0].boxes contains detection info for this frame
        boxes_data = results[0].boxes

        # Process each detection and feed into the appropriate TVM
        for i in range(len(boxes_data)):
            cls_id   = int(boxes_data.cls[i])           # Class index (integer)
            cls_name = model.names[cls_id]              # Class name string
            conf     = float(boxes_data.conf[i])        # Confidence score [0,1]
            xywh     = boxes_data.xywhn[i].tolist()     # Normalised [x,y,w,h]

            detection = {
                "class": cls_name,
                "confidence": conf,
                "box": xywh
            }

            if cls_name in tvm_map:
                tvm_map[cls_name].add_detection(detection)

        # Get stable output from each TVM and draw on frame
        for cls_name, tvm in tvm_map.items():
            stable = tvm.get_stable_detection()
            if stable:
                frame = draw_detection(
                    frame,
                    stable["box"],
                    stable["class"],
                    stable["confidence"],
                    width, height
                )
```

```
126
127         # Write annotated frame to output video
128         writer.write(frame)
129
130         # Show progress every 50 frames
131         if frame_idx % 50 == 0:
132             print(f"Processed␣{frame_idx}/{total}␣frames")
133
134     # Release resources
135     cap.release()
136     writer.release()
137     print(f"Output␣saved␣to␣{OUTPUT_PATH}")
138
139
140 if __name__ == "__main__":
141     main()
```

30 evaluate.py — Evaluation Script

```
1  # evaluate.py
2  # Purpose: Evaluate model performance and compare TVM ON vs TVM
       OFF.
3
4  import time
5  import cv2
6  import numpy as np
7  import matplotlib.pyplot as plt
8  from ultralytics import YOLO
9  from tvm import TemporalVerificationModule
10
11 MODEL_PATH   = "models/traffic_light_v1/weights/best.pt"
12 TEST_VIDEO   = "input_video.mp4"
13 CONF_THRESH  = 0.4
14 WINDOW_SIZE  = 5
15
16 def run_detection(video_path, model, use_tvm=True):
17     """
18 ␣␣␣␣Run␣detection␣on␣a␣video.
19 ␣␣␣␣Returns␣per-frame␣confidence␣scores␣and␣class␣labels.
20 ␣␣␣␣"""
21     cap = cv2.VideoCapture(video_path)
22     tvm = TemporalVerificationModule(window_size=WINDOW_SIZE) if
          use_tvm else None
23
24     frame_times = []    # For FPS calculation
25     raw_labels  = []    # Labels without TVM
26     tvm_labels  = []    # Labels with TVM
27
28     while True:
29         ret, frame = cap.read()
```

```
        if not ret:
            break

        t0 = time.time()

        results = model.predict(source=frame, conf=CONF_THRESH,
           verbose=False)
        boxes   = results[0].boxes

        best_label = None
        best_conf  = 0.0
        best_box   = None

        for i in range(len(boxes)):
            conf = float(boxes.conf[i])
            if conf > best_conf:
                best_conf  = conf
                best_label = model.names[int(boxes.cls[i])]
                best_box   = boxes.xywhn[i].tolist()

        t1 = time.time()
        frame_times.append(t1 - t0)

        raw_labels.append(best_label)

        if tvm and best_label:
            tvm.add_detection({"class": best_label, "confidence":
                best_conf,
                               "box": best_box})
            stable = tvm.get_stable_detection()
            tvm_labels.append(stable["class"] if stable else None
               )
        else:
            tvm_labels.append(best_label)

    cap.release()
    fps = 1.0 / np.mean(frame_times) if frame_times else 0
    return raw_labels, tvm_labels, fps


def count_flickers(labels):
    """
    Count how many times the label changes between consecutive
       frames.
    A lower count = more stable = better.
    """
    changes = 0
    for i in range(1, len(labels)):
        if labels[i] != labels[i - 1]:
            changes += 1
    return changes
```

```python


def plot_stability(raw_labels, tvm_labels):
    """Plot raw vs TVM label changes over time."""

    # Map labels to integers for plotting
    label_map = {"red": 0, "green": 1, "yellow": 2, None: -1}
    raw_int = [label_map.get(l, -1) for l in raw_labels]
    tvm_int = [label_map.get(l, -1) for l in tvm_labels]

    frames = list(range(len(raw_labels)))

    plt.figure(figsize=(12, 4))
    plt.plot(frames, raw_int, label="Raw (No TVM)", alpha=0.6,
        color="red")
    plt.plot(frames, tvm_int, label="TVM Stabilised", alpha=0.9,
        color="green")
    plt.yticks([-1, 0, 1, 2], ["None", "Red", "Green", "Yellow"])
    plt.xlabel("Frame Number")
    plt.ylabel("Detected Class")
    plt.title("Detection Stability: Raw vs TVM")
    plt.legend()
    plt.tight_layout()
    plt.savefig("output/plots/stability_comparison.png", dpi=150)
    plt.show()
    print("Stability plot saved.")


def main():
    model = YOLO(MODEL_PATH)

    print("Running without TVM...")
    raw_labels, _, fps_raw = run_detection(TEST_VIDEO, model,
        use_tvm=False)

    print("Running with TVM...")
    _, tvm_labels, fps_tvm = run_detection(TEST_VIDEO, model,
        use_tvm=True)

    raw_flickers = count_flickers(raw_labels)
    tvm_flickers = count_flickers(tvm_labels)

    print("\n===== Evaluation Results =====")
    print(f"FPS (no TVM):         {fps_raw:.1f}")
    print(f"FPS (with TVM):       {fps_tvm:.1f}   (TVM overhead is
        small)")
    print(f"Label flickers (raw): {raw_flickers}")
    print(f"Label flickers (TVM): {tvm_flickers}")
    print(f"Stability improvement: {100*(raw_flickers-
        tvm_flickers)/max(raw_flickers,1):.1f}%")

```

```
    import os
    os.makedirs("output/plots", exist_ok=True)
    plot_stability(raw_labels, tvm_labels)

    # Run built-in YOLO validation for mAP metrics
    print("\nRunning YOLO validation for mAP metrics...")
    val_results = model.val(data="data/data.yaml", split="test",
        verbose=True)
    print(f"mAP@0.5:      {val_results.box.map50:.4f}")
    print(f"mAP@0.5:0.95: {val_results.box.map:.4f}")
    print(f"Precision:    {val_results.box.mp:.4f}")
    print(f"Recall:       {val_results.box.mr:.4f}")


if __name__ == "__main__":
    main()
```

Part VI
Mathematical Deep Dive

31 Confidence Averaging — Worked Example

31.1 Formula

$$\bar{C}_t = \frac{\sum_{i=0}^{N-1} w_{t-i} \cdot C_{t-i}}{\sum_{i=0}^{N-1} w_{t-i}}$$

Symbol meanings:

- $\bar{C}_t$: averaged confidence at time t (what we compute).
- N: window size (e.g. $N = 5$).
- C_{t-i}: confidence from i frames ago.
- w_{t-i}: weight for that frame. We use $w = [1, 2, 3, 4, 5]$ for $N = 5$ (most recent = weight 5).

31.2 Numerical Example

Last 5 confidence scores (oldest first): [0.55, 0.62, 0.70, 0.78, 0.85]
Weights (oldest first): [1, 2, 3, 4, 5], sum = 15.

$$\bar{C}_t = \frac{1 \cdot 0.55 + 2 \cdot 0.62 + 3 \cdot 0.70 + 4 \cdot 0.78 + 5 \cdot 0.85}{15} = \frac{0.55 + 1.24 + 2.10 + 3.12 + 4.25}{15} = \frac{11.26}{15} \approx 0.75$$

The simple (unweighted) average would be $\frac{0.55+0.62+0.70+0.78+0.85}{5} = 0.70$. The weighted average is higher because recent frames (which are stronger detections here) carry more weight.

32 IoU — Manual Calculation

32.1 Setup

Ground truth box: top-left $(100, 100)$, bottom-right $(200, 200)$. Area $= 100 \times 100 = 10{,}000$.

Predicted box: top-left $(150, 150)$, bottom-right $(250, 250)$. Area $= 100 \times 100 = 10{,}000$.

32.2 Overlap Region

$$x_{\text{left}} = \max(100, 150) = 150, \quad x_{\text{right}} = \min(200, 250) = 200$$

$$y_{\text{top}} = \max(100, 150) = 150, \quad y_{\text{bottom}} = \min(200, 250) = 200$$

Intersection area $= (200 - 150) \times (200 - 150) = 50 \times 50 = 2{,}500$.

32.3 Union

$$\text{Union} = 10{,}000 + 10{,}000 - 2{,}500 = 17{,}500$$

32.4 IoU

$$\text{IoU} = \frac{2{,}500}{17{,}500} \approx 0.143$$

This is less than 0.5, so this prediction would be counted as a **False Positive**.

33 Precision and Recall — Worked Example

33.1 Scenario

In a test video with 100 traffic lights, our model:

- Correctly detects 80 of them (TP = 80).
- Detects 10 things that aren't traffic lights (FP = 10).
- Misses 20 real traffic lights (FN = 20).

33.2 Calculations

$$\text{Precision} = \frac{80}{80 + 10} = \frac{80}{90} \approx 0.889$$

$$\text{Recall} = \frac{80}{80 + 20} = \frac{80}{100} = 0.800$$

Interpretation: 88.9% of our detections were correct (high precision). We found 80% of all real traffic lights (good recall).

34 Bounding Box Smoothing — Worked Example

34.1 Problem

The x-coordinate of a traffic light bounding box over 5 frames: $[0.42, 0.44, 0.40, 0.43, 0.41]$.

34.2 Moving Average

$$\bar{x} = \frac{0.42 + 0.44 + 0.40 + 0.43 + 0.41}{5} = \frac{2.10}{5} = 0.42$$

Instead of using the raw $x = 0.41$ (noisy), we draw the box at $x = 0.42$ (stable).

Part VII
Testing and Results

35 Running Inference on a Test Video

```
# Make sure your virtual environment is activated first
# Place your test video as: input_video.mp4

python detect.py
# Output is saved to output/output_video.mp4
```

36 Interpreting Results

- **mAP@0.5 ¿ 0.80**: Very good. The model detects most traffic lights correctly.
- **mAP@0.5 between 0.60–0.80**: Acceptable. Consider collecting more data or training more epochs.
- **mAP@0.5 ¡ 0.60**: Needs improvement. Check data quality, annotations, or class balance.
- **FPS ¿ 25**: Real-time capable.
- **Flicker reduction ¿ 40%**: TVM is working effectively.

37 Common Errors and Fixes

Error	Cause	Fix
`FileNotFoundError: best.pt`	Training not completed	Run `train.py` first
`CUDA out of memory`	Batch size too large	Reduce `batch=8` or use `device="cpu"`
`No module named ultralytics`	Library not installed	Run `pip install ultralytics`
`Cannot open video`	Wrong file path	Check `VIDEO_PATH` in `detect.py`
Boxes all wrong size	Wrong image size	Ensure `imgsz=640` matches training

Part VIII
Report Writing Guide

38 Structure of the Final Report

1. **Abstract** (150–200 words): What problem, what method, what results.
2. **Introduction**: Motivate the problem. Why traffic light detection matters. Project scope.
3. **Literature Review**: Brief mention of YOLO versions, temporal consistency methods.
4. **Methodology**: Describe the pipeline (Part 2 of this guide). Include a system architecture diagram.
5. **Dataset**: Where data came from, how many images, class distribution (include a bar chart).
6. **Experiments**: Training settings, hyperparameters, hardware used.
7. **Results**: Table of mAP, Precision, Recall, FPS. Figure of TVM ON vs OFF stability. Qualitative results (screenshots of detected frames).
8. **Discussion**: What worked well. What didn't. Limitations.
9. **Conclusion**: Summary of contributions. Future work.
10. **References**: Cite YOLOv8 paper, dataset papers, OpenCV.

39 Essential Figures and Tables

- System architecture block diagram (draw in LaTeX or include as PNG).

- Training loss curves (generated automatically by Ultralytics in `runs/` folder).
- Precision–Recall curve.
- Sample output frames showing bounding boxes.
- Comparison table: Raw detection vs TVM-stabilised (flickers, FPS).

40 Sample Results Table

Method	**Precision**	**Recall**	**mAP@0.5**	**FPS**
YOLOv8s (no TVM)	0.87	0.81	0.83	42
YOLOv8s + TVM	0.89	0.84	0.85	40

Glossary of Technical Terms

Anchor box A predefined bounding box shape used in YOLO to help predict object locations.

Backbone The part of a neural network that extracts features from the raw input image.

Backpropagation Algorithm for adjusting neural network weights based on prediction errors.

Batch size Number of images processed simultaneously during one training step.

Bounding box A rectangle drawn around a detected object, described by (x, y, w, h).

CNN Convolutional Neural Network — a neural network that uses sliding filters to process images.

Confidence score A number $\in [0, 1]$ indicating how certain the model is about a detection.

Convolution A mathematical operation that slides a filter across an image to produce a feature map.

Data augmentation Artificially increasing dataset size by flipping, rotating, or colour-jittering images.

Deque A Python data structure (double-ended queue) that efficiently removes oldest items when full.

Epoch One complete pass through the entire training dataset.

Feature map Output of a convolutional layer — highlights specific patterns in the input.

FPN Feature Pyramid Network — merges features at different scales for better small-object detection.

FPS Frames Per Second — how many video frames a system can process each second.

Ground truth The correct, human-annotated label for an image used to evaluate model accuracy.

HSV Hue-Saturation-Value colour space, useful for colour-based filtering under varying lighting.

IoU Intersection over Union — a metric for how well a predicted box overlaps the ground truth box.

Label The correct class name associated with an annotated object (e.g. "red", "green").

Learning rate Controls how large each weight update step is during training.

mAP Mean Average Precision — the primary evaluation metric for object detectors.

Majority voting Choosing the most frequently occurring class label across multiple frames.

NMS Non-Maximum Suppression — removes duplicate bounding boxes, keeping the highest-confidence one.

Normalisation Scaling pixel values from $[0, 255]$ to $[0.0, 1.0]$ for stable training.

OpenCV Open Source Computer Vision Library — Python library for image/video processing.

Overfitting When a model memorises training data but fails on new data.

Pixel The smallest unit of a digital image, storing RGB colour values.

Precision Fraction of detections that are correct: $TP/(TP + FP)$.

ReLU Rectified Linear Unit — activation function: $f(x) = \max(0, x)$.

Recall Fraction of real objects that are detected: $TP/(TP + FN)$.

Resolution The number of pixels in an image, expressed as width × height.

RGB Red-Green-Blue colour model — three channels each ranging from 0 to 255.

Sliding window A fixed-size memory buffer that stores the most recent N frames.

TVM Temporal Verification Module — the system that stabilises detections across frames.

Transfer learning Starting from a model pretrained on a large dataset and fine-tuning on a smaller one.

Virtual environment An isolated Python installation to avoid library conflicts between projects.

Weight A learnable number in a neural network that controls connection strength between neurons.

YOLO You Only Look Once — a fast, one-stage object detection algorithm.